Contents

Syllabus **1**
- Aim . . . 1
- Learning outcomes . . . 1
- Structure . . . 1
- Teaching method . . . 3
- Assessment . . . 3
- Environment . . . 4
- Excluded from scope . . . 4

Syllabus

Technologies for Artificial Intelligence Università degli Studi dell'Aquila — MSc in Computer Science, first year 30 hours, 10 lessons of 3 hours · Autumn 2026 Instructor: Fabio Antonini

---

Aim

To give students a working command of the fundamentals of machine learning: the methods, the mathematics underneath them, and — above all — the experimental discipline needed to tell a result that holds from one that only appears to.

The course is deliberately **not** a survey of current AI products. It covers the foundations on which those products rest, in enough depth that a graduate can read a paper, implement a method, and judge whether an evaluation is sound.

Learning outcomes

On completion, a student can:

1. Frame a practical problem as a supervised or unsupervised learning task, and say when machine learning is the wrong tool.
2. Prepare a real dataset: exploration, missing values, outliers, scaling, encoding, and feature construction — without leaking information from the test set.
3. Derive and implement the core algorithms from first principles: least squares, logistic regression, margin maximisation, information gain, backpropagation.
4. **Design an honest experiment**: choose a validation scheme, tune hyperparameters without contaminating the estimate, and report results with their uncertainty.
5. Select an appropriate model family for a problem and justify the choice against alternatives.
6. Diagnose a failing model — distinguishing bias, variance, leakage, and a badly posed problem.
7. Communicate findings, including limitations, to a technical audience.

Structure

#	Lesson	Topics
1	**Introduction and the ML workflow**	What learning from data means; supervised, unsupervised and self-supervised; the end-to-end workflow; environment setup; a first complete example; where ML fails, and its societal risks
2	**Data: exploration and preparation**	Exploratory analysis; missing values and outliers; scaling; categorical encoding; feature engineering; scikit-learn pipelines; first encounter with data leakage
3	**Regression**	Linear regression from scratch and with scikit-learn; the cost function; gradient descent; multiple and polynomial regression; Ridge and Lasso; overfitting
4	**Classification and evaluation metrics**	Logistic regression; decision boundaries; the confusion matrix; precision, recall, F1; ROC and AUC; class imbalance; multiclass strategies
5	**Experimental methodology**	Train/validation/test; cross-validation; the bias-variance decomposition; learning curves; hyperparameter search; **data leakage in depth**; reproducibility
6	**k-NN, Naive Bayes and SVM**	k-nearest neighbours and the curse of dimensionality; Naive Bayes and its independence assumption; support vector machines: margins and the kernel trick
7	**Trees and ensembles**	Decision trees; entropy, Gini and information gain; bagging and random forests; boosting and XGBoost; feature importance and interpretability

#	Lesson	Topics
8	**Unsupervised learning**	k-means and its objective; hierarchical clustering; DBSCAN; PCA via eigendecomposition and SVD; t-SNE; anomaly detection
9	**Neural networks**	From the perceptron to the multilayer network; what a hidden layer does; how a network learns (backpropagation in pictures, one step by hand); TensorFlow/Keras; softmax; early stopping and dropout; the vanishing gradient. The advanced version of the lesson is kept, not examined, in `Advanced/`
10	**Convolutional networks and synthesis**	Convolution and pooling; data augmentation; transfer learning; synthesis of the course and where the field goes next

Lesson 5 sits at the centre of the course by design. Students who can program will produce models that appear to work within weeks. Without honest validation they will produce invalid results with great confidence, and everything after Lesson 5 depends on their being able to tell the difference.

Teaching method

Roughly 60% hands-on, 40% theory. Each lesson combines a lecture with live work in Jupyter notebooks. Every method is implemented from scratch at least once before the library version is introduced.

Three kinds of material per lesson:

- **Slides** — the lecture skeleton.
- **Handout** — the reference text, with the complete mathematical derivations.
- **Notebooks** — runnable implementations.

Plus a self-check quiz and an assessed weekly exercise.

Assessment

One of the ten weekly exercises is drawn at random at the exam, and the student talks through their own notebook for it. There is no written paper and no separate project; the exercises are the whole assessment.

They are not collected or marked week by week. Each is set at the end of a lesson and discussed at the start of the next, and the one drawn at the exam is where the mark comes from — so keep all ten notebooks, and treat none of them as optional.

Criterion	Weight
Methodological correctness	40%
Implementation	30%
Interpretation and communication	30%

See `../Assessment/` for the exam structure and what the discussion asks of you.

Environment

All work happens in a preconfigured Docker image with Python, JupyterLab, scikit-learn, XGBoost and TensorFlow. No local Python installation is needed.

Nothing in this course requires a paid service, an API key or a network call at runtime. Every notebook runs offline once the environment is built.

See `PREREQUISITES.md` and `Setup/Docker_Quickstart.md`.

Excluded from scope

Large language models, transformers, retrieval-augmented generation and agent architectures are **not** covered: they belong to a separate course in the programme. `BRIDGE.md` sets out what carries forward instead: which of this course's results the next one reuses unchanged, and which of its habits nobody will teach again there because they are assumed.